A

Mini Project

report On

"LifeChain – Permissioned Blockchain-Based Blood Bank Traceability System"

Submitted by

1. Vishal Bhor VU1F2324041
2. Member 2 Name Roll No.
3. Member 3 Name Roll No.

Under guidance of

Dr. Gayatri Bachhav

Department of Computer Engineering

Vasantdada Patil Pratishthan's College of Engineering & Visual Arts

Sion, Mumbai - 400 022

University Of Mumbai

2026-27

INSTITUTE VISION & MISSION

VISION:

To provide an environment to educate, encourage and explore students by facilitating research, entrepreneurship, employability to achieve social and professional goals.

MISSION:

1. To foster entrepreneurship & strengthen industry institute interaction to enhance career opportunities for the employability of students.
2. To encourage collaborations with industries and academic institutes in terms of projects & internship by creating area for Research and Development.
3. To build up appropriate moral and ethical skills and to promote holistic development of students through various academic, technical, social and cultural activities.

COMPUTER ENGINEERING DEPARTMENT VISION & MISSION

VISION:

To develop a center of excellence in computer engineering and produce globally competent engineers who contribute towards the progress of the engineering community and society as a whole.

MISSION:

1. To provide students with diversified engineering knowledge to work in a multidisciplinary environment.
2. To provide a platform to cultivate research, innovation, and entrepreneurial skills.
3. To produce world-class computer engineering professionals with moral values and leadership abilities for the sustainable development of society.

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

PEO1: To create graduates with sound fundamental knowledge of computer engineering.

PEO2: To enhance students' skills towards emerging technologies to propose solutions for engineering problems and entrepreneurial pursuits, making them employable.

PEO3: To produce technology professionals with ethical values and commitment to lifelong learning.

PROGRAM SPECIFIC OUTCOMES (PSOs)

1. Graduates of programme will be able to provide effective and efficient real time solutions using practical knowledge in Computer Engineering domain.

2. Graduate of programme will be able to use engineering practices, strategies and tactics for the development, operation and maintenance of software system.

CERTIFICATE

This is to certify that the mini project entitled “LifeChain – Permissioned Blockchain-Based Blood Bank Traceability System” of Fourth Year computer engineering have been satisfactorily completed for academic year 2026-27 and it is bonafide work of the following students

1. Vishal Bhor VU1F2324041
2. Member 2 Name Roll No.
3. Member 3 Name Roll No.

Dr. Gayatri Bachhav
Project Guide

Dr. Gayatri Bachhav
Project Coordinator

Dr. Rais Mulla
HOD Computer Engineering

Dr. Alam N. Shaikh
Principal

DECLARATION

We hereby affirm that the work presented in this report is exclusively our own creation. The Project entitled- “LifeChain – Permissioned Blockchain-Based Blood Bank Traceability System” is fully designed/ developed by us and every part of the project is original work and it is not copied/ purchased from any source.

Vishal Bhor VU1F2324041 .....

Member 2 Name Roll No. .....

Member 3 Name Roll No. .....

Date :
Place : Mumbai

MINI PROJECT APPROVAL

This Mini Project entitled "LifeChain – Permissioned Blockchain-Based Blood Bank Traceability System" is approved for the Fourth Year Computer Engineering for following students

Vishal Bhor (VU1F2324041)
Member 2 Name (Roll No.)
Member 3 Name (Roll No.)

EXAMINERS

1 _____
Internal Examiner Name and Sign

2 _____
External Examiner Name and Sign

Date :
Place : Mumbai

ACKNOWLEDGEMENT

We are highly grateful to the Dr. Alam N. Shaikh, Principal, Vasantdada Patil Pratishthan's College of Engineering and Visual Arts, Chunabhatti, Mumbai, for providing this opportunity to carry out the mini project.

The constant guidance and encouragement received from Dr. Rais A. Mulla, H.O.D. Computer Engineering Department, VPPCOE & VA, Mumbai has been of great help in carrying out the project work and is acknowledged with reverential thanks.

We would like to express a deep sense of gratitude and thanks profusely to Dr. Gayatri Bachhav without her wise counsel and able guidance, it would have been impossible to complete the project in this manner.

We express gratitude to other faculty members of computer engineering department of VPPCOE & VA for their intellectual support throughout the course of this work.

Finally, we are indebted to all who so ever have contributed in this report work.

1. Vishal Bhor
2. Member 2 Name
3. Member 3 Name

ABSTRACT

Blood cannot be manufactured; every unit comes from a donor and passes through several organizations before it reaches a patient. A blood bank collects it, a laboratory screens it, and a hospital stores and transfuses it. Each of these steps is usually recorded in a separate register or software system, so it is difficult to know at any moment where a unit is, whether it has been tested, and who received it. When a donor is later found to be infected or a patient has a transfusion reaction, tracing the affected units across organizations is slow, and paper or spreadsheet records can be edited without leaving a trace. LifeChain is an educational permissioned blockchain application that tracks every blood unit from donor to patient and shows how a blockchain network can make blood bank management safe, transparent and auditable.

The network is modelled on Hyperledger Fabric and has five member organizations: a blood bank, a testing laboratory, two hospitals and a health regulator. Each organization has its own certificate authority and an ECDSA P-256 identity registered with a Membership Service Provider (MSP). Business rules are written as chaincode named bloodtrace, with the functions CollectBlood, RecordTest, TransferUnit, ReceiveUnit, TransfuseUnit and DiscardUnit. Every transaction follows the execute–order–validate model. The submitting organization's peer and the regulator's peer simulate the chaincode and sign the read/write set, the ordering service packages endorsed transactions into SHA-256 hash-linked blocks with a Merkle root, and peers validate endorsement signatures and key versions (MVCC) before committing. The chaincode enforces role-based permissions and blood-safety rules: untested, reactive, expired or ABO/Rh-incompatible units cannot be issued or transfused. Donor and patient identities stay off-chain and only SHA-256 pseudonymous references are stored.

The application is implemented in Node.js with no external packages, and is operated through a web interface with screens for donation, lab testing, distribution, hospital use, inventory, unit tracing, a block explorer, the network configuration and security demonstrations. Ten automated tests confirm the access rules, safety checks, double-allocation prevention and tamper detection. The network runs on a single machine, with all peers simulated in one process, so it is a demonstration of the concepts rather than a production deployment.

Keywords: Permissioned blockchain, Hyperledger Fabric, chaincode, endorsement policy, MSP, SHA-256, ECDSA, Merkle tree, MVCC, blood bank management, traceability.

TABLE OF CONTENTS

ABSTRACT 1
CHAPTER 1 -- INTRODUCTION 3
CHAPTER 2 -- LITERATURE SURVEY 5
CHAPTER 3 -- PROPOSED SYSTEM AND DESIGN 6
CHAPTER 4 -- EXPERIMENTAL SETUP AND IMPLEMENTATION 10
CHAPTER 5 -- RESULTS AND DISCUSSION 13
CHAPTER 6 -- CONCLUSION AND FUTURE SCOPE 23
REFERENCES 24

CHAPTER 1 -- INTRODUCTION

1.1 Background

Blood transfusion saves lives in surgery, trauma, childbirth and the treatment of diseases such as thalassemia and cancer. Because blood can only come from donors and every component has a limited shelf life (platelets last only five days), blood banks must manage collection, screening, storage and distribution carefully. A single unit is handled by several independent organizations, and each keeps its own records. Hospitals usually cannot see the real-time stock of other centres, units expire unused in one place while another place faces a shortage, and when a quality problem is discovered, investigators must collect and compare records from every organization by hand.

A blockchain is a shared ledger in which records, once written, cannot be quietly changed. In a **permissioned** blockchain, only known and approved organizations can join the network, each member is identified by a certificate issued by its own certificate authority, and smart contracts (called chaincode in Hyperledger Fabric) decide which member may perform which action. This makes permissioned blockchains well suited to healthcare consortia, where participants are known institutions and data must not be publicly visible.

1.2 Problem Statement

There is no single, tamper-evident record that all participants trust and that shows the complete journey of a blood unit from donor to patient. Records are fragmented across organizations, safety rules depend on manual checks, and edits leave no audit trail. This project builds a permissioned blockchain application in which the blood bank, laboratory, hospitals and regulator record every action on a blood unit as a signed, endorsed transaction, so that the current status and full history of each unit are always available and safety rules are enforced automatically.

1.3 Objectives

- Design a permissioned network of five organizations, each with its own MSP identity (ECDSA P-256 key pair and certificate details).
- Write the bloodtrace chaincode with the functions CollectBlood, RecordTest, TransferUnit, ReceiveUnit, TransfuseUnit and DiscardUnit, each restricted to the responsible organization.
- Implement the execute–order–validate flow with the endorsement policy AND('<CreatorOrg>.peer', 'RegulatorMSP.peer'), hash-linked blocks and MVCC validation.
- Enforce blood-safety rules on-chain: mandatory screening for five infections, shelf-life expiry and ABO/Rh compatibility at transfusion.
- Keep donor and patient personal data off-chain by storing only SHA-256 pseudonymous references.
- Build a web interface with dashboard, role screens, inventory, unit trace, block explorer, network view and security tests.

- Demonstrate that double allocation of a unit is prevented and that any tampering with a stored block is detected.

1.4 Scope

The application is intended for educational demonstration. All five peers, the ordering service and the MSP are simulated inside one Node.js process on a single computer, and the ledger is saved as JSON files in the data folder. It does not connect to a real Hyperledger Fabric network, real hospital systems or barcode scanners, and it has no user login beyond choosing the organization identity in the interface. Donor and patient registration with personal details is outside the scope; only pseudonymous references are used.

1.5 Motivation

The project brings together the core ideas of permissioned blockchains in one practical, socially useful workflow: identity and membership, chaincode-based access control, endorsement, ordering into hash-linked blocks, Merkle roots, validation and the difference between the blockchain and the world state. Blood bank traceability is a problem where these properties have a clear benefit: a shared history that no single organization can alter, automatic enforcement of safety rules, and fast tracing of units during recalls.

1.6 Organization of Report

Chapter 1 introduces the problem and objectives. Chapter 2 reviews related approaches. Chapter 3 describes the network architecture, chaincode design and workflows. Chapter 4 documents the software environment and implementation. Chapter 5 presents results, test cases and discussion. Chapter 6 summarizes the project, limitations and future enhancements.

CHAPTER 2 -- LITERATURE SURVEY

Blood bank record-keeping approaches generally fall into four groups: manual registers, centralized blood bank management software, public blockchains and permissioned blockchains. Each balances convenience, trust, privacy and cost differently.

Approach	Typical capability	Strength	Limitation
Manual registers / spreadsheets	Each organization records donations, tests and issues in its own books	Familiar, no infrastructure needed	Slow tracing; no real-time stock view; entries can be altered without trace
Centralized blood bank software / portals	One database for stock and requests (for example national portals such as e-RaktKosh [7])	Real-time stock search; standard formats	Single point of failure and control; whoever runs the database can change records; limited cross-organization audit
Public blockchain (e.g. Ethereum)	Record events in a smart contract on an open network	Public, tamper-evident, always available	Anyone can join and read; transaction fees; health data would be publicly visible
Permissioned blockchain (e.g. Hyperledger Fabric [1], [2])	Known organizations run peers and chaincode with endorsement policies	Identity-based access, privacy, no mining fees, shared audit trail	Needs consortium governance and node infrastructure
LifeChain	Five-organization network, bloodtrace chaincode, web interface	Safety rules enforced on-chain; full unit history; tamper and double-allocation detection	Simulated network in one process; local demonstration only

2.1 Research Gap and Project Position

Many studies propose blockchain for healthcare supply chains, but most remain at the architecture level and do not show how endorsement, ordering and validation actually work on the data. LifeChain is a compact academic prototype rather than a production platform. Its contribution is an end-to-end, readable implementation of the Fabric transaction flow applied to blood units, from identities and chaincode to blocks, validation codes and a user interface, together with built-in demonstrations of the properties that make the approach useful: role-based control, safety enforcement, conflict prevention and tamper evidence.

2.2 Summary

A useful educational prototype should keep the workflow understandable, make the trust model explicit and be honest about limits. LifeChain is designed around these principles.

CHAPTER 3 -- PROPOSED SYSTEM AND DESIGN

3.1 Proposed System

LifeChain consists of a permissioned blockchain network of five organizations, the bloodtrace chaincode, a client gateway (REST API) and a browser interface. A user selects the organization identity they act for and performs an action, for example registering a donation. The gateway creates a transaction proposal signed with that organization's private key and sends it to the peers required by the endorsement policy. Each endorsing peer runs the chaincode on its copy of the world state, producing a read/write set, and signs the result. The ordering service then places the endorsed transaction in a new block linked to the previous block by its hash. Finally each peer validates the endorsements and the versions of the keys read, commits the block, and updates the world state for valid transactions. The interface shows the transaction ID, block number, endorsements and validation code.

3.2 System Architecture

Layer	Responsibilities
User / Browser	Choose the MSP identity, fill in forms, view stock, history and blocks
Web interface (public/index.html, app.js, style.css)	Ten screens: Dashboard, Collect Donation, Lab Testing, Distribution, Hospital, Inventory, Trace Unit, Block Explorer, Network, Security Tests
Client gateway (src/server.js)	Node.js HTTP server; REST API; passes the caller's identity (X-Org header) to the ledger; returns 403 for access errors
Peers and ledger (src/ledger.js)	MSP identities and signatures, proposal simulation, endorsement, ordering service, block creation, validation, commit, world state and history queries
Chaincode (src/chaincode.js)	bloodtrace business logic: role checks, safety rules, unit state changes
Storage (data/)	ledger.json (all blocks) and msp.json (organization keys and certificates); the world state is rebuilt from the blocks at start-up

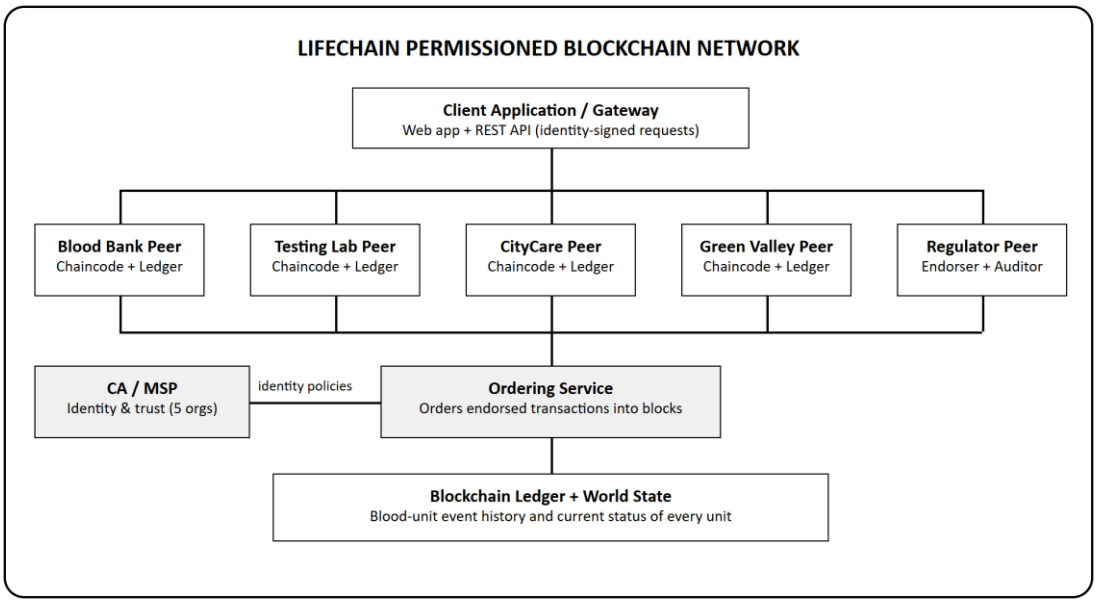


Figure 3.1: LifeChain permissioned blockchain network architecture

Personal details of donors and patients never reach the ledger. The chaincode converts the entered donor or patient ID into a pseudonymous reference (REF- followed by the first 12 hexadecimal characters of its SHA-256 hash). The regulator is a full member: its peer endorses every transaction, but its identity has read-only access and cannot submit changes.

3.3 Main Modules

Module	Purpose	Key operations
Membership (MSP)	Identify organizations	Generate an ECDSA P-256 key pair, CA name and certificate serial for each organization; sign and verify with SHA-256
Endorsement	Execute chaincode safely	Simulate the proposal on each endorsing peer, record read versions and writes, compare results, sign the read/write set
Ordering service	Create blocks	Queue endorsed transactions, compute the Merkle root (data hash), link to the previous block hash
Validation and commit	Accept only correct transactions	Check endorsement policy signatures and MVCC versions; mark VALID, ENDORSEMENT_POLICY_FAILURE or MVCC_READ_CONFLICT; update world state
bloodtrace chaincode	Business rules	Collect, test, transfer, receive, transfuse and discard blood units with role and safety checks
Queries	Read the ledger	Current units, unit history from blocks, statistics by blood group, full chain validation
Web interface	User interaction	Role screens, receipts with endorsements, provenance timeline, block explorer, security demonstrations

3.4 Chaincode Design

The bloodtrace chaincode stores one record per blood unit in the world state under the key UNIT_<unitId>.

Field	Type	Description
unitId	string	Unique bag barcode, format BU-… (e.g. BU-2026-0113)
donorRef	string	Pseudonymous donor reference (REF- + 12 hex characters of SHA-256)
bloodGroup	string	One of A+, A-, B+, B-, AB+, AB-, O+, O-
component / volumeMl	string / number	Whole Blood, Packed RBC, Platelets or Plasma; 50–500 ml
collectedAt / expiryDate	timestamp	Expiry = collection time + shelf life (35, 42, 5 or 365 days)
testResult	object	HIV, HBV, HCV, Syphilis, Malaria results, verdict SAFE / UNSAFE, tested by and at
status	string	COLLECTED, AVAILABLE, REJECTED, IN_TRANSIT, AT_HOSPITAL, TRANSFUSED or DISCARDED
custodian / transferTo	string	Organization currently holding the unit / hospital it is addressed to
transfusion	object	Pseudonymous patient reference, patient blood group, hospital and time
lastAction / lastUpdatedBy	string	Latest event and the organization that recorded it

Function	Access	Behaviour
CollectBlood(unitId, donorId, bloodGroup, component, volumeMl)	BloodBankOrg	Validates input; rejects duplicate unit IDs; stores the unit as COLLECTED with expiry and donor reference
RecordTest(unitId, results)	LabOrg	Requires status COLLECTED and all five results; AVAILABLE if all negative, otherwise REJECTED
TransferUnit(unitId, toOrg)	Current custodian	Only AVAILABLE, non-expired units, only to a registered hospital; status IN_TRANSIT
ReceiveUnit(unitId)	Addressed hospital	Requires IN_TRANSIT; hospital becomes custodian; status AT_HOSPITAL
TransfuseUnit(unitId, patientId, patientBloodGroup)	Custodian hospital	Blocks expired and incompatible units; records pseudonymous patient; status TRANSFUSED
DiscardUnit(unitId, reason)	Current custodian	Records disposal of expired or reactive units; status DISCARDED
History / query (gateway)	All members incl. regulator	Current state, complete unit history from the blocks, chain validation

Validation code	Meaning
VALID	Endorsements satisfy the policy and all keys read are unchanged; writes are applied to the world state
ENDORSEMENT_POLICY_FAILURE	A required endorsement is missing or its signature does not verify
MVCC_READ_CONFLICT	A key read during simulation was changed by an earlier transaction; the transaction stays in the block but its writes are not applied

3.5 Donation and Testing Workflow

1. The blood bank user signs in as LifeLine Blood Bank (BloodBankMSP) and enters the unit ID, donor ID, blood group, component, volume and collection centre.
2. The gateway creates and signs a CollectBlood proposal. The BloodBankMSP and RegulatorMSP peers simulate it and sign the read/write set.
3. The ordering service creates a new block and the peers validate and commit it; the unit is now COLLECTED.
4. The laboratory user (LabMSP) selects the unit and records the five screening results with RecordTest. If all are negative the unit becomes AVAILABLE; any positive result makes it REJECTED.

3.6 Distribution and Transfusion Workflow

1. The blood bank issues an AVAILABLE, non-expired unit to a hospital with TransferUnit; the unit becomes IN_TRANSIT.
2. Only the addressed hospital can confirm receipt with ReceiveUnit; it becomes the custodian and the unit is AT_HOSPITAL.
3. When transfusing, the hospital enters the patient ID and blood group. The chaincode checks expiry and red-cell or plasma compatibility; an incompatible request is rejected before ordering.
4. A compatible transfusion is committed and the unit becomes TRANSFUSED, with a pseudonymous patient reference.

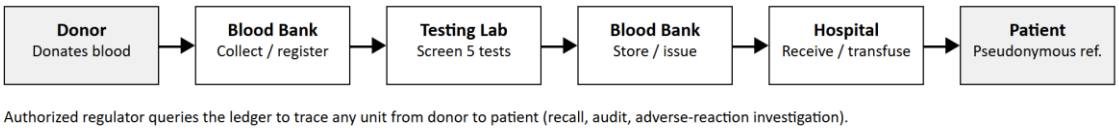


Figure 3.2: Movement and status changes of a blood unit

3.7 Trace and Audit Workflow

1. Any member, including the regulator, enters a unit ID on the Trace Unit screen.
2. The gateway reads the current state and scans the blocks for every transaction that wrote the unit's key.
3. The page shows the provenance timeline (collected, tested, dispatched, received, transfused) with block numbers, submitting organizations, endorsers and validation codes.

CHAPTER 4 -- EXPERIMENTAL SETUP AND IMPLEMENTATION

4.1 Software Requirements

Component	Technology	Purpose
Runtime	Node.js 18 or later (tested on 22.19.0)	Runs the network simulation, chaincode and gateway
Cryptography	Node.js crypto module	SHA-256 hashing, ECDSA P-256 key generation, signing and verification
Web server / API	Node.js http module (no frameworks)	REST API and static file server at http://localhost:3000
Interface	HTML, CSS, JavaScript (Fetch API)	Single-page application with ten screens
Storage	JSON files (data/ledger.json, data/msp.json)	Persistent block store and organization identities
Testing	Node.js built-in test runner (node --test)	Ten automated tests
Editor / browser	VS Code; Chrome or Edge	Development and demonstration

The project has **no external npm dependencies**, so it runs immediately after cloning, without npm install.

4.2 Hardware Requirements

A standard laptop or desktop with a modern browser is sufficient. The simulated network, five peers and ordering service run in a single process and use very little memory. No internet connection is required after the project files are copied.

4.3 Project Structure

```
LifeChain-BloodBank-MiniProject/
   src/chaincode.js        bloodtrace chaincode (business rules)
   src/ledger.js           MSP, endorsement, ordering, validation, world
state
   src/server.js           REST API gateway and static web server
   public/index.html, app.js, style.css   web interface
   scripts/seed.js         creates a realistic sample history
   scripts/reset.js        clears the ledger
   test/lifechain.test.js  automated tests
   data/ledger.json, msp.json   blocks and identities (created at run time)
   package.json
```

4.4 Identities and Signatures (MSP)

When the ledger is first created, an ECDSA key pair on the P-256 (prime256v1) curve is generated for each organization, together with its CA name (for example ca.bloodbank.lifechain.com), a random certificate serial and its peer name. These are stored in data/msp.json. A request from an identity that is not enrolled is rejected with an access error. The client's proposal is signed with the submitting organization's private key, and each endorsing peer signs the transaction ID together with the canonical (key-sorted) JSON of the read/write set. During validation and full-chain checks these signatures are verified with the public keys.

4.5 Ledger and Chaincode Implementation

The ledger keeps two structures, as a Fabric peer does. The **blockchain** is an array of blocks; each block header holds its number, the previous block's hash and the data hash (Merkle root of the SHA-256 hashes of its transactions), and the block hash is the SHA-256 of the header. Block 0 is a configuration (genesis) block listing the organizations, orderer, chaincode and endorsement policy. The **world state** maps each key to its latest value and a version (block number:transaction number); it is rebuilt by replaying all valid transactions when the server starts.

During simulation the chaincode accesses data only through getState and putState. getState records the version of every key read, and putState collects writes without changing the ledger. A chaincode error, such as an access-denied or incompatible-transfusion error, stops the proposal at the endorsement stage, so it is never ordered. Role checks use a requireOrg helper that compares the submitting organization with the organizations allowed for the action, and returns messages such as "Access denied: CityCareOrg is not permitted to register blood donations".

4.6 Seed Script

scripts/seed.js deletes any old ledger and identities, creates a new network, and records a realistic history: 12 donations of different blood groups and components (one block), screening of 11 units in which unit BU-2026-0107 tests positive for HBV (one block), and ten further transactions for transfers, receipts, transfusions and the discard of the reactive unit. The result is a valid chain of 13 blocks and 33 transactions, which is shown in the terminal and used for the screenshots.

4.7 Automated Tests

The test file uses Node.js's built-in test runner and assert module. Each test creates a fresh ledger in a temporary folder, so the project data is never touched. It contains ten test cases:

- The genesis block is created and the chain is valid.
- The blood bank can collect a unit, and it is endorsed by BloodBankMSP and RegulatorMSP with a pseudonymous donor reference.
- Organizations cannot act outside their role (hospital collecting, blood bank testing, regulator transferring, unknown identity).
- Duplicate unit IDs are rejected.
- A positive screening result rejects the unit and blocks its issue.
- Untested units cannot be issued, and only the addressed hospital can receive a unit.
- An incompatible transfusion is blocked, a compatible one is recorded, and the history shows all five events.
- Expired units cannot be issued.
- Double allocation: of two conflicting transfers in one block, only the first is VALID and the second gets MVCC_READ_CONFLICT.
- Tampering with a stored block is detected, and restoring the ledger makes the chain valid again.

4.8 Running the Project

No installation is needed beyond Node.js. Open a terminal in the project folder:

```
npm run seed      # optional: create the sample history (13 blocks)
npm test          # run the 10 automated tests
npm start         # start the network gateway
npm run reset     # optional: clear the ledger and start fresh
```

Then open http://localhost:3000 in a browser and choose an organization identity in the sidebar.

CHAPTER 5 -- RESULTS AND DISCUSSION

The following figures document the principal stages of the working project, captured from our own run of the application.

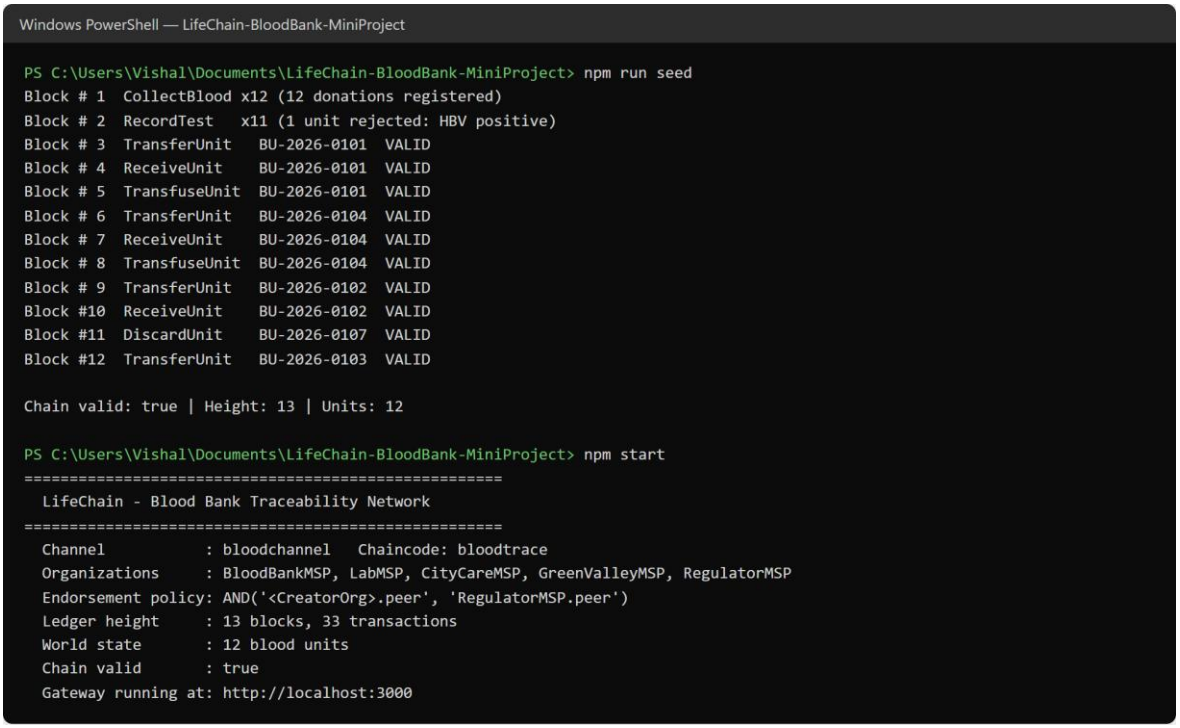


Figure 5.1: Seeding the ledger and starting the network

The seed script prints each block as it is committed; all transactions are VALID and the chain is valid with 13 blocks and 12 units. The gateway then prints the channel, organizations, endorsement policy and ledger height.

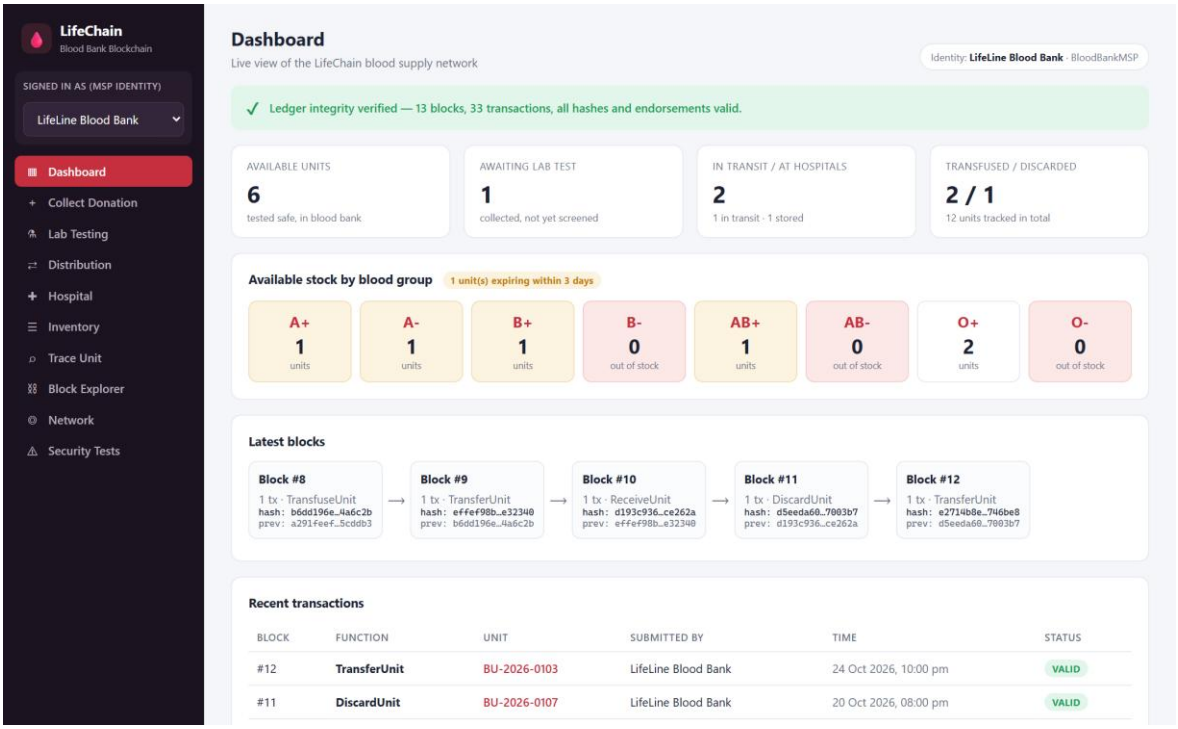


Figure 5.2: Dashboard

The dashboard confirms ledger integrity and shows the number of available units, units awaiting tests, units in transit and transfused or discarded units, available stock for each blood group with expiry warnings, the latest blocks and the recent transactions.

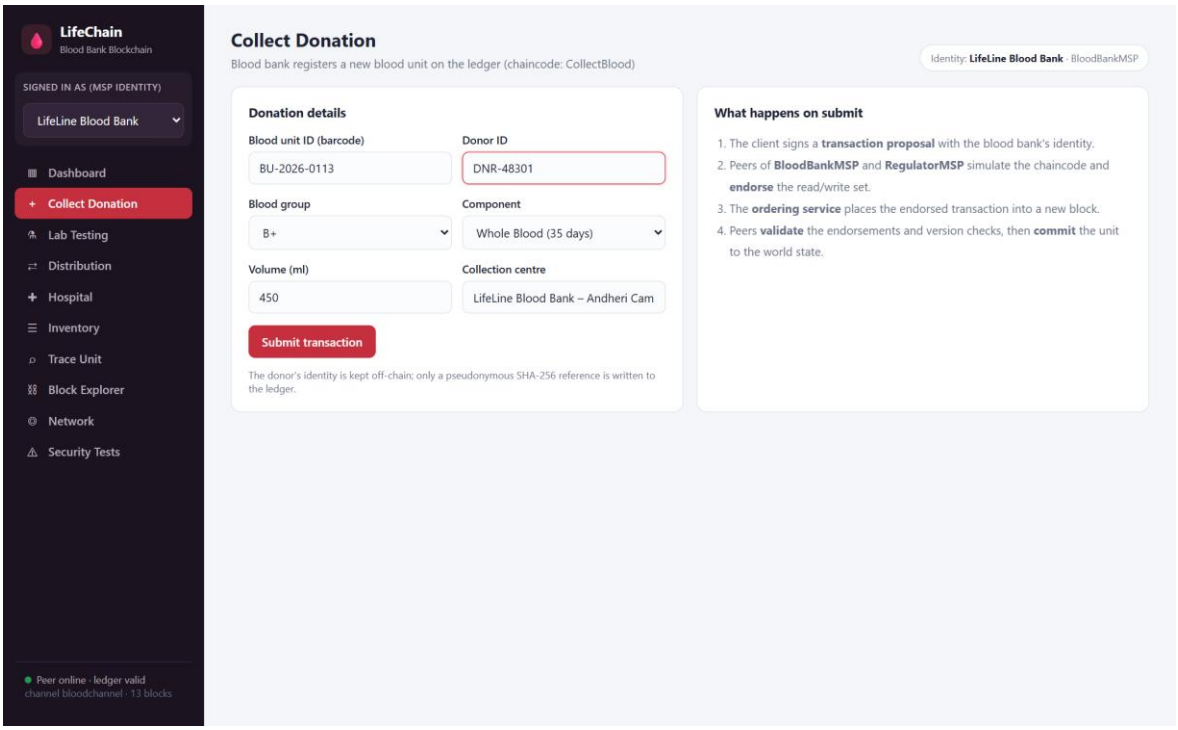


Figure 5.3: Collect donation form (blood bank)

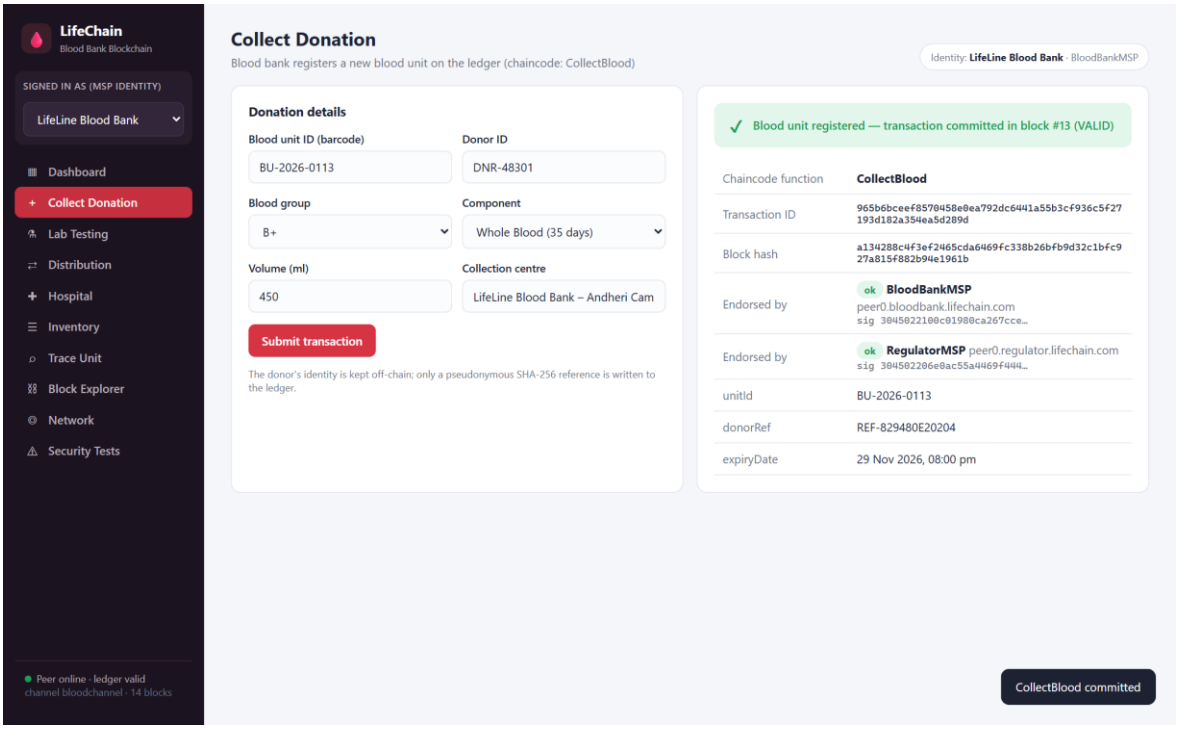


Figure 5.4: Donation committed to the ledger

After CollectBlood is committed, the receipt shows the transaction ID, block number and hash, the endorsements by BloodBankMSP and RegulatorMSP with their signatures, the pseudonymous donor reference and the calculated expiry date.

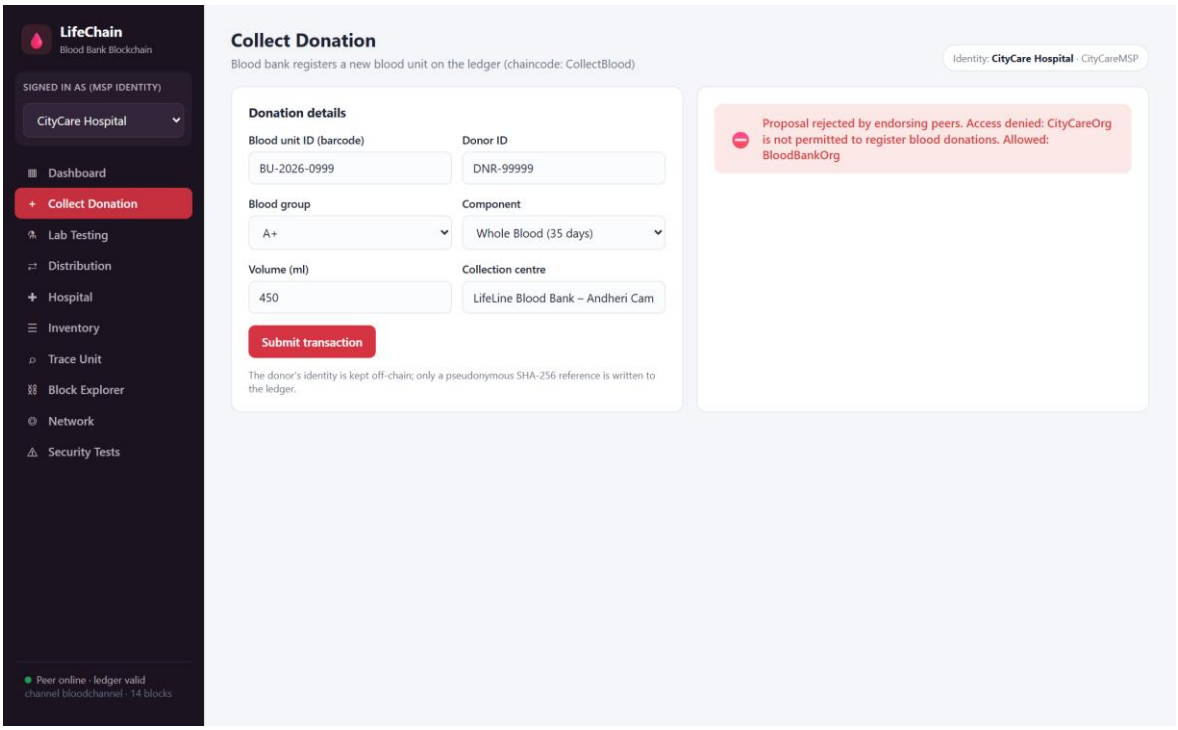


Figure 5.5: Access denied – hospital trying to register a donation

When the identity is switched to CityCare Hospital and the same form is submitted, the endorsing peers reject the proposal: only BloodBankOrg may register donations. The transaction never reaches the ordering service.

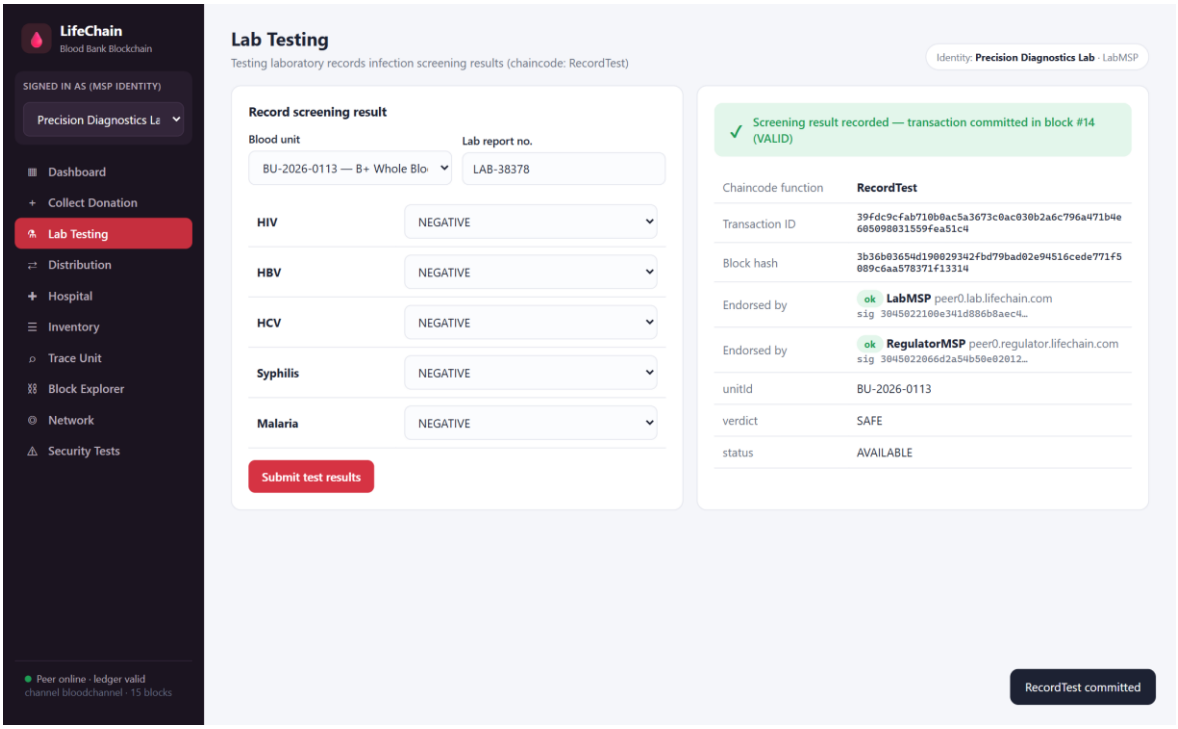


Figure 5.6: Laboratory screening result recorded

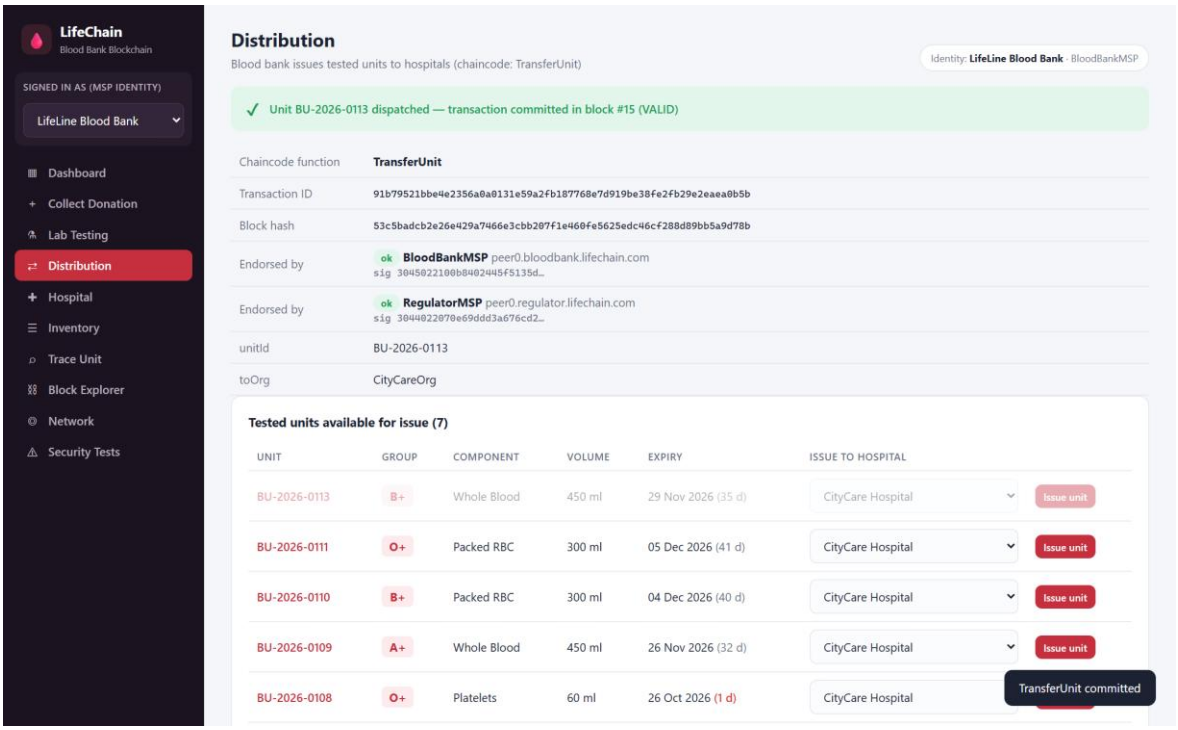


Figure 5.7: Unit issued to a hospital

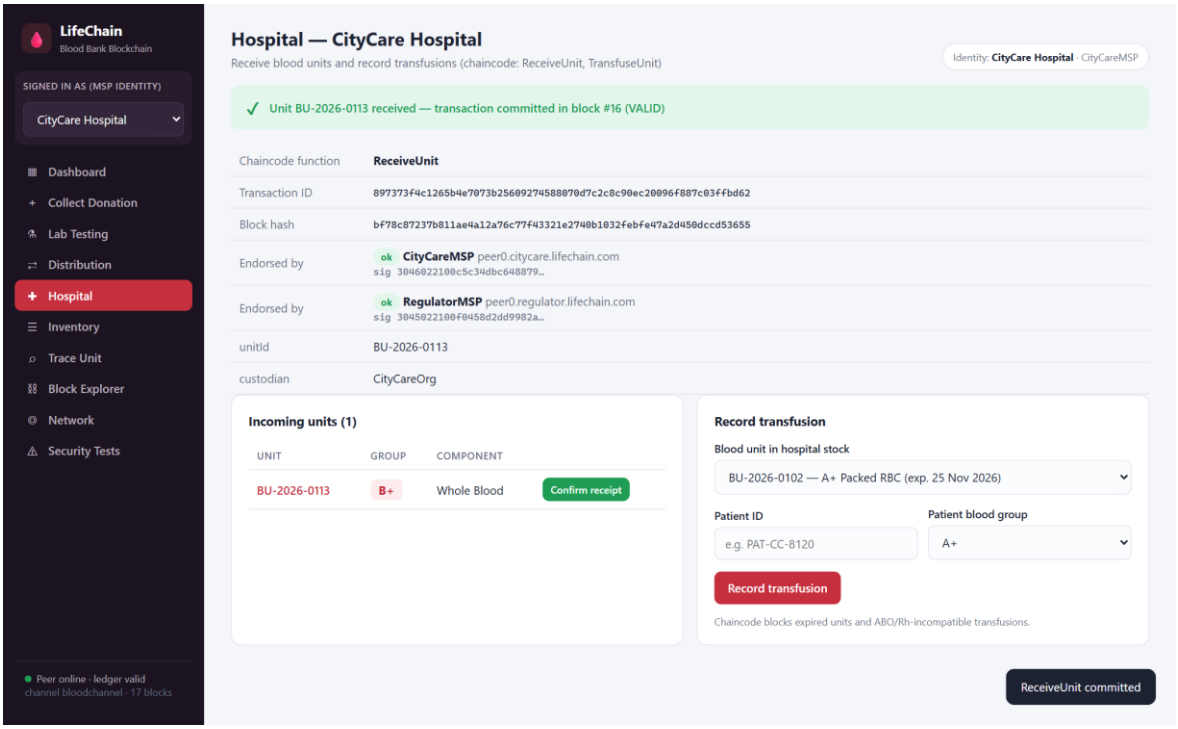


Figure 5.8: Hospital confirms receipt

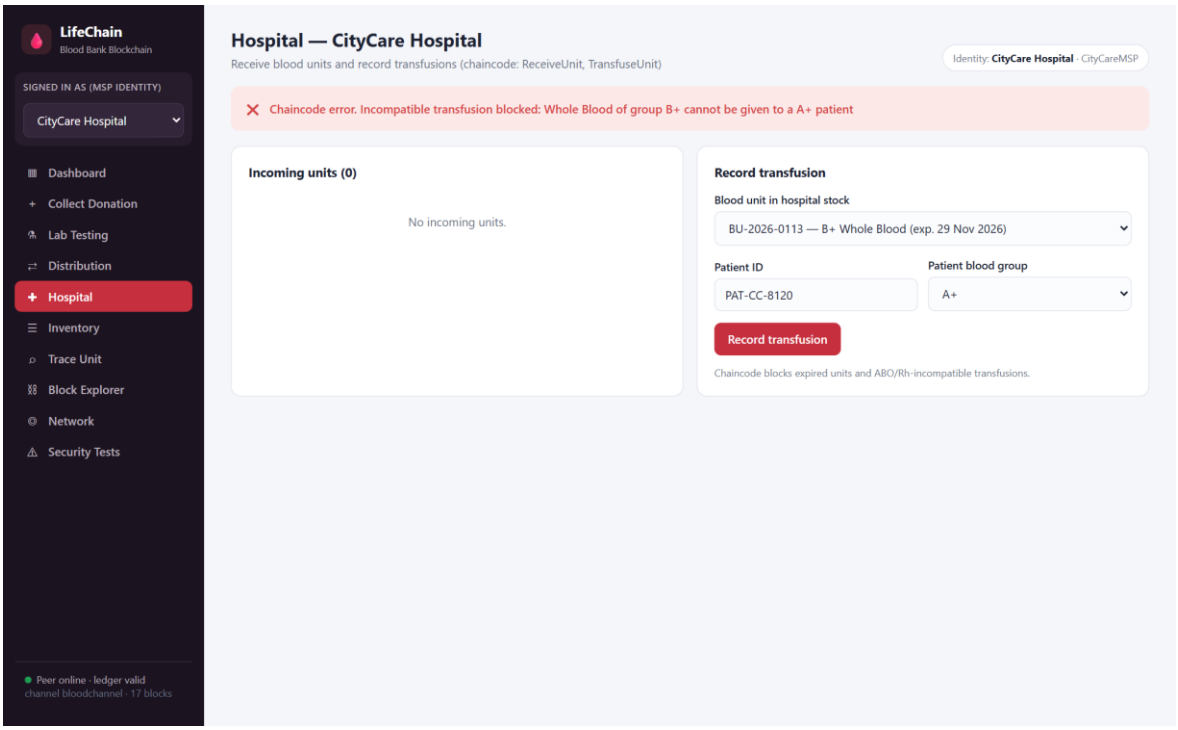


Figure 5.9: Incompatible transfusion blocked

A B+ whole-blood unit cannot be given to an A+ patient. The chaincode rejects the request with a clear message, so an unsafe transfusion cannot be recorded.

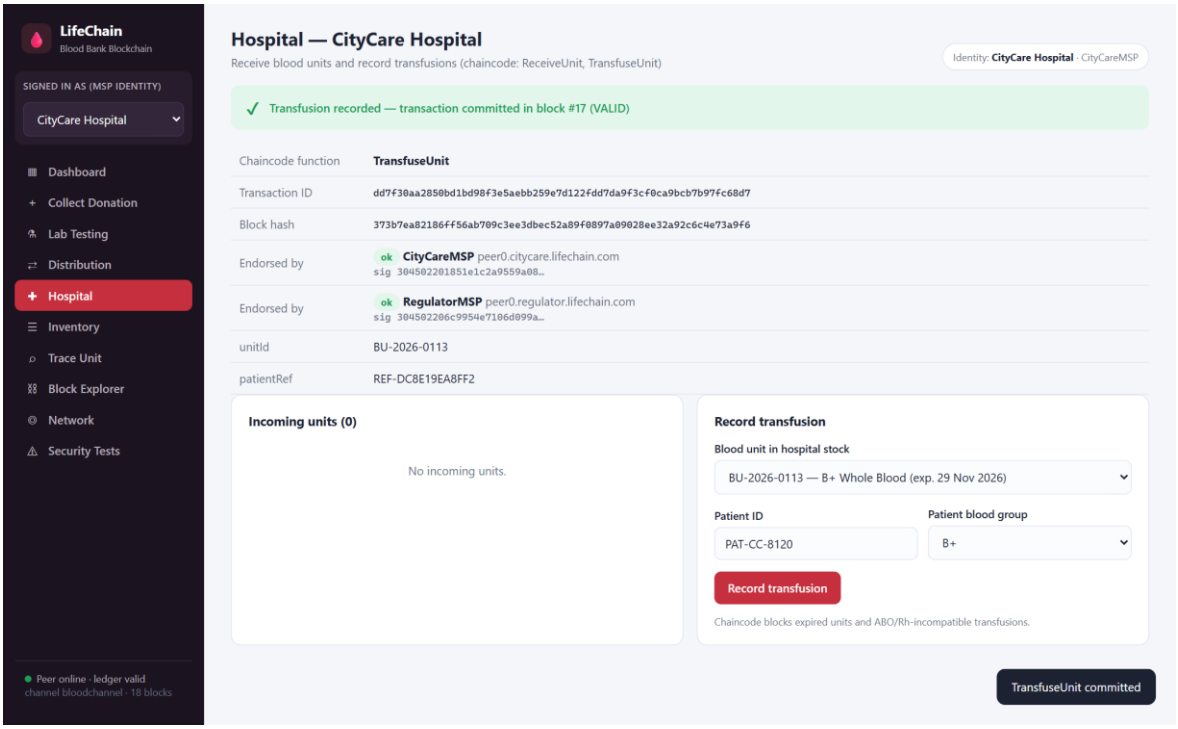


Figure 5.10: Compatible transfusion recorded

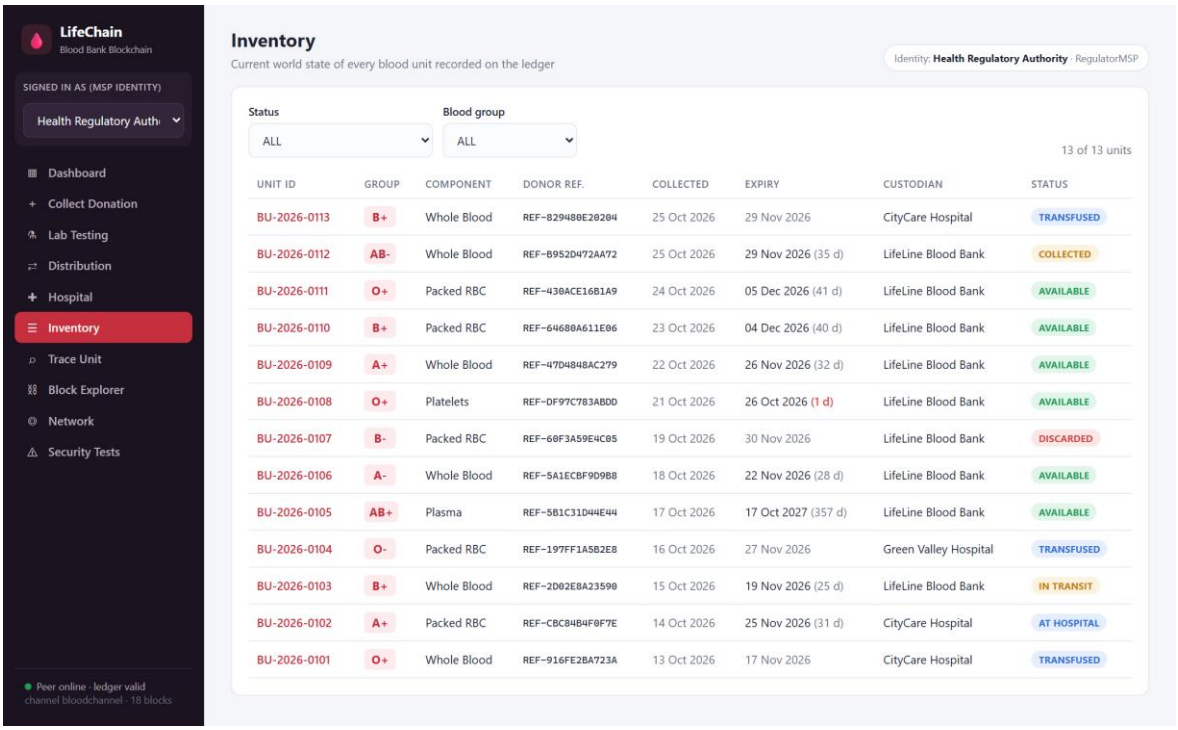


Figure 5.11: Inventory (world state) viewed by the regulator

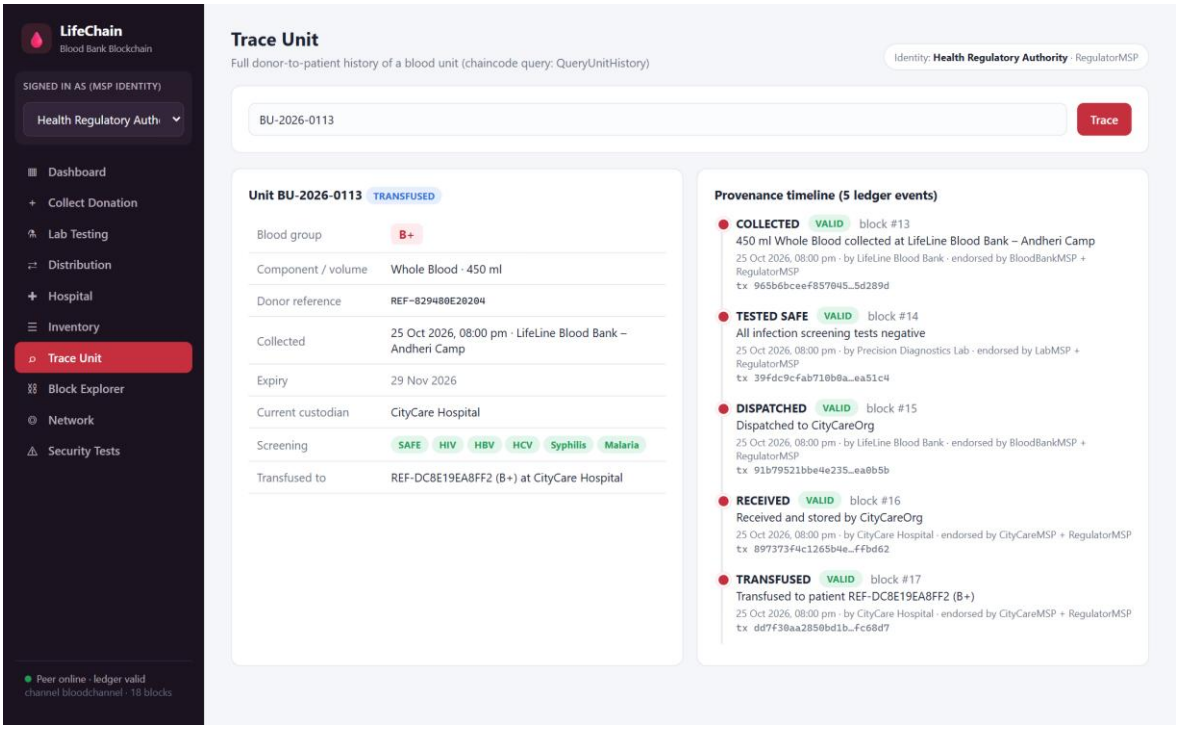


Figure 5.12: Donor-to-patient trace of unit BU-2026-0113

The trace screen shows the complete provenance of the unit: collected, tested safe, dispatched, received and transfused, each with its block number, submitting organization, endorsers and validation code.

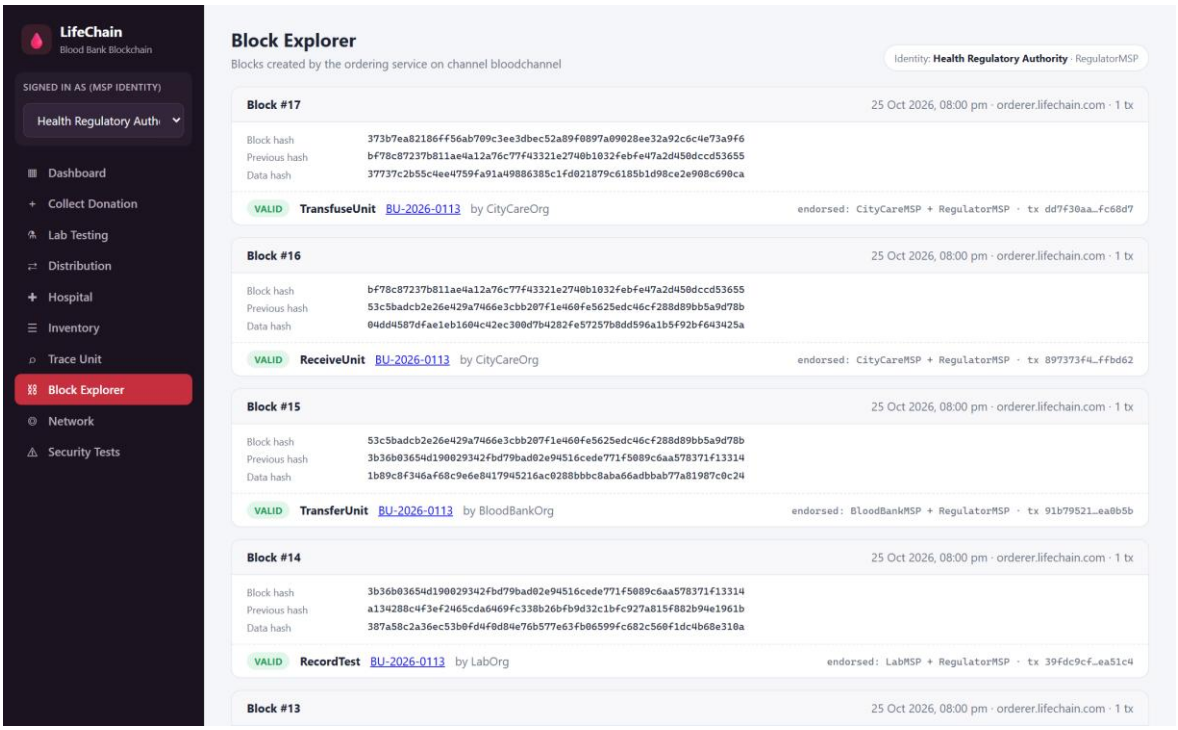


Figure 5.13: Block explorer

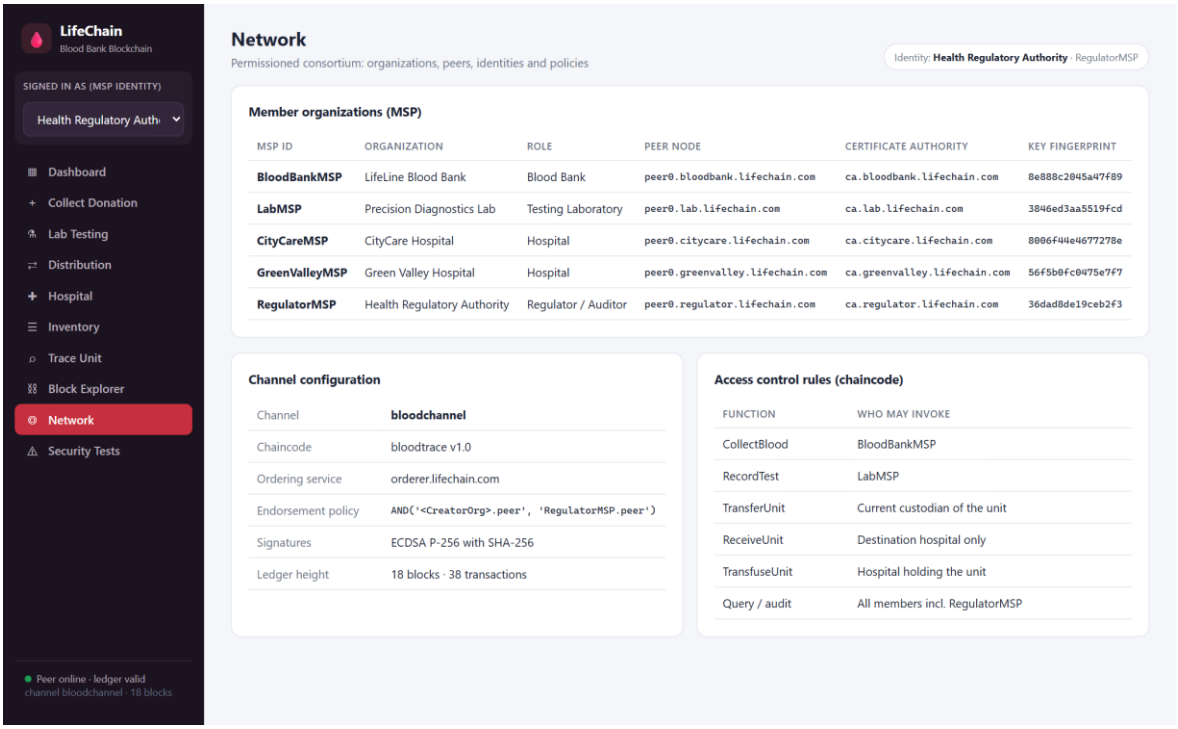


Figure 5.14: Network members, channel configuration and access rules

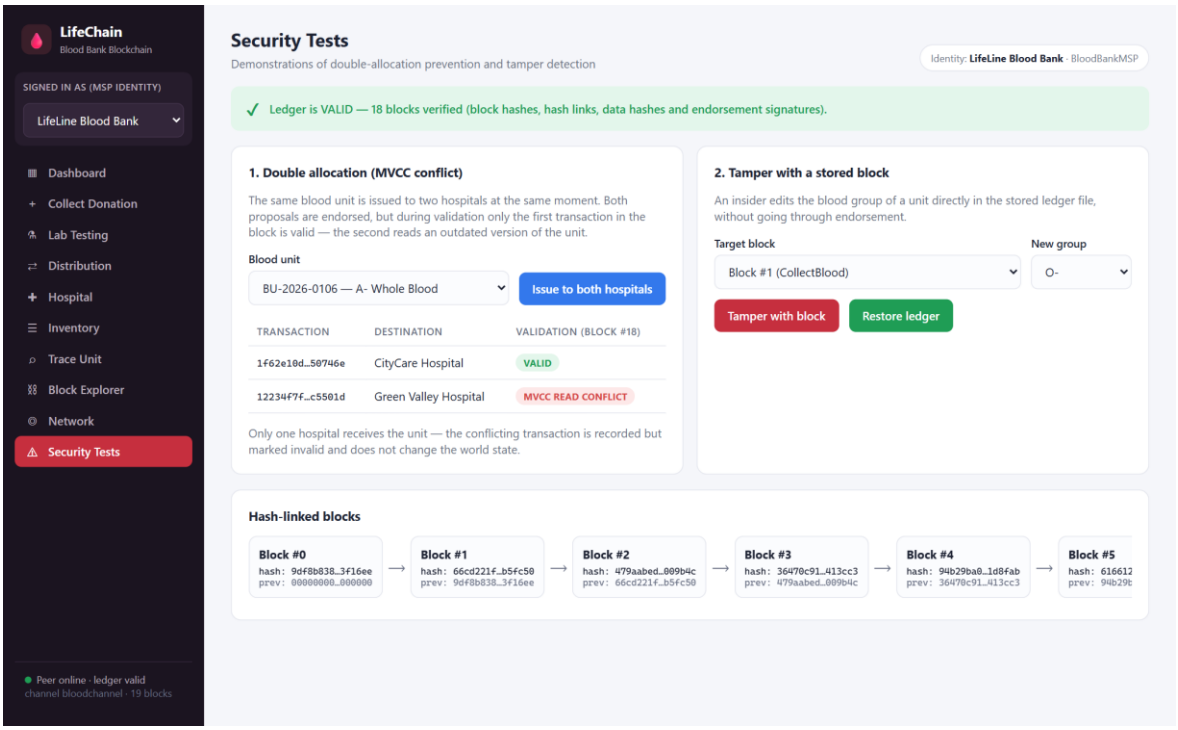


Figure 5.15: Double-allocation test – MVCC read conflict

Two transfers of the same unit to different hospitals are ordered into one block. The first is VALID; the second fails with MVCC_READ_CONFLICT, because the unit's version changed, so the unit can never be issued twice.

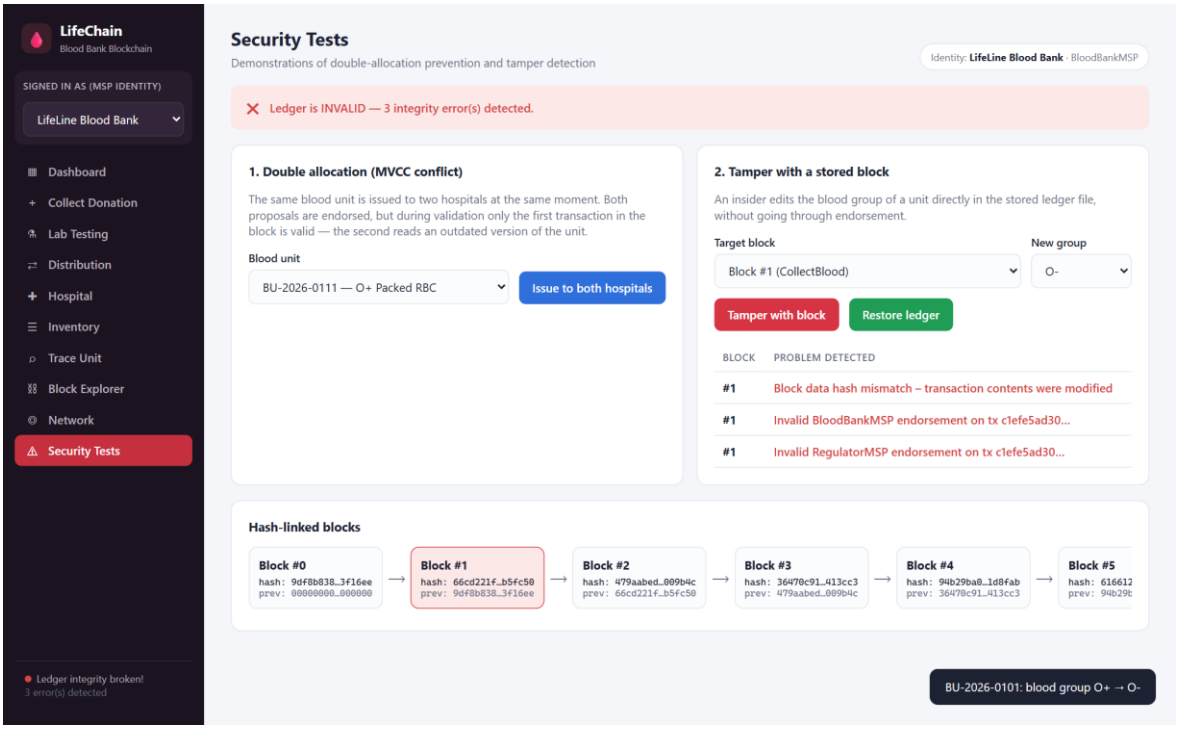


Figure 5.16: Tamper test – modified block detected

Changing the blood group stored in block #1 directly in the ledger breaks the block data hash and the endorsement signatures. The integrity check reports the errors and the block is highlighted in the hash-linked chain until the ledger is restored.

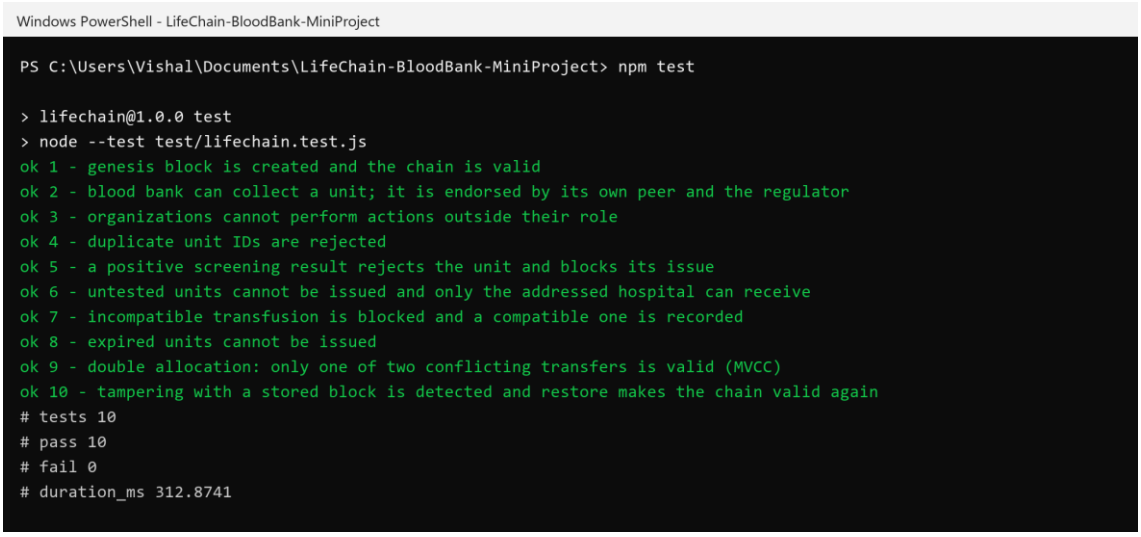


Figure 5.17: Automated test results (10 of 10 passed)

5.1 Security Observations

Property	How it is achieved
Permissioned membership	Only identities enrolled in an organization's MSP can submit transactions; unknown identities are rejected
Role-based access	Each chaincode function checks the submitting organization (blood bank, lab, custodian or addressed hospital); the regulator is read-only
Endorsement policy	Every transaction must be signed by the submitting organization's peer and the regulator's peer, AND('<CreatorOrg>.peer', 'RegulatorMSP.peer')
Tamper evidence	Blocks are linked by SHA-256 hashes, transactions are summarized by a Merkle root and endorsements are ECDSA signatures; any modification is detected
No double allocation	MVCC validation rejects a transaction whose read keys changed after simulation
Blood safety	Untested, reactive or expired units cannot be issued, and ABO/Rh-incompatible transfusions are blocked
Privacy	Donor and patient identities are stored only as SHA-256 pseudonymous references
Auditability	The full history of every unit, including rejected (invalid) transactions, remains in the blocks

5.2 Discussion

The results show that the main properties of a permissioned blockchain can be demonstrated clearly on a real-world healthcare problem. Every organization works on the same ledger, yet each can only perform the actions it is responsible for, and the regulator's endorsement on every transaction gives independent oversight. Safety rules are checked by the chaincode on every endorsing peer, so they cannot be skipped by one user. The trace screen reduces a recall investigation that would otherwise require collecting records from several organizations to a single query.

The demonstration also has clear limits. All peers run in one process and share one copy of the code, so it does not show real network failures, separate machines or a consensus protocol such as Raft. Each organization has a single identity, the browser chooses the identity through a header without real authentication, and the private keys are stored unencrypted in a local file. Pseudonymous references are SHA-256 hashes without a salt, so a short donor ID could be guessed by hashing candidate IDs. These points are acceptable for a learning prototype but would need to be addressed in a real deployment.

CHAPTER 6 -- CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

LifeChain demonstrates a complete permissioned blockchain workflow for blood bank management. Five organizations with their own MSP identities share one ledger; the bloodtrace chaincode records collection, screening, distribution, receipt, transfusion and disposal of blood units with role-based access and blood-safety rules; and every change passes through endorsement by the submitting organization and the regulator, ordering into hash-linked blocks, and MVCC validation. A web interface lets users perform each step, trace any unit from donor to patient, inspect blocks and run security demonstrations, and ten automated tests confirm the expected behaviour.

The project meets its academic objective of showing how a blockchain network can make blood bank management safe, transparent and auditable. Its most important limitation is that the network is simulated in a single process, so it is a learning prototype and not a production system.

6.2 Limitations

- All peers, the ordering service and the MSP run in one Node.js process on one machine; there is no real distributed consensus.
- Each organization has one identity selected in the interface, with no user login or role hierarchy within an organization.
- Private keys are stored unencrypted in data/msp.json.
- Pseudonymous references are unsalted SHA-256 hashes, which can be guessed for short or predictable IDs.
- There is no integration with barcode/RFID scanners, cold-chain temperature sensors or existing hospital systems.
- The interface is not optimized for mobile devices and has not been load-tested.

6.3 Future Scope

- Deploy the chaincode on a real Hyperledger Fabric network with separate peers, a Raft ordering service and Fabric CA.
- Add user-level identities and attribute-based access control within each organization.
- Use salted or keyed hashes (HMAC) for donor and patient references and private data collections for sensitive fields.
- Record cold-chain temperature readings from IoT sensors during storage and transport.
- Generate QR codes on blood bags that link to the unit’s trace page.
- Add donor notifications, stock alerts and inter-hospital requests for rare blood groups.

6.4 Final Note

The project should be treated as a learning prototype. Its most valuable next step is moving from the simulated network to a real multi-node permissioned network with proper identity and key management.

REFERENCES

[1] Hyperledger Foundation, “Hyperledger Fabric documentation.” https://hyperledger-fabric.readthedocs.io/ (accessed October 2026).

[2] E. Androulaki et al., “Hyperledger Fabric: A distributed operating system for permissioned blockchains,” in Proc. 13th EuroSys Conference, 2018.

[3] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. https://bitcoin.org/bitcoin.pdf

[4] National Institute of Standards and Technology, “FIPS 180-4: Secure Hash Standard (SHS).” https://csrc.nist.gov/pubs/fips/180-4/upd1/final (accessed October 2026).

[5] National Institute of Standards and Technology, “FIPS 186-5: Digital Signature Standard (DSS),” 2023.

[6] R. C. Merkle, “A digital signature based on a conventional encryption function,” in Advances in Cryptology – CRYPTO ’87, 1988.

[7] Ministry of Health and Family Welfare, Government of India, “e-RaktKosh: Centralized blood bank management system.” https://eraktkosh.mohfw.gov.in/ (accessed October 2026).

[8] World Health Organization, “Blood safety and availability” fact sheet. https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability (accessed October 2026).

[9] OpenJS Foundation, “Node.js documentation: Crypto and Test runner modules.” https://nodejs.org/api/ (accessed October 2026).

[10] LifeChain project files: chaincode.js, ledger.js, server.js, seed.js, lifechain.test.js, index.html, app.js and style.css.